Learn Go with Tests (Excerpt)

Learn Go with Tests -- Go Fundamentals (Excerpt)
Hello, World
    How it works
    How to test
    Go modules?
    Back to Testing
        Writing tests
        Go's documentation
        Hello, YOU
        A note on source control
        Constants
    Hello, world... again
        Back to source control
        Discipline
    Keep going! More requirements
        French
    `switch`
        one...last...refactor?
    Wrapping up
        Some of Go's syntax around
        The TDD process and *why* the steps are important
Integers
    Write the test first
    Try and run the test
    Write the minimal amount of code for the test to run and check the failing test output
    Write enough code to make it pass
    Refactor
        Testable Examples
    Wrapping up
Iteration
    Write the test first
    Try and run the test
    Write the minimal amount of code for the test to run and check the failing test output
    Write enough code to make it pass
    Refactor
        Benchmarking
    Practice exercises
    Wrapping up
Arrays and slices
    Write the test first
    Try to run the test
    Write the minimal amount of code for the test to run and check the failing test output
    Write enough code to make it pass
    Refactor
        Arrays and their type
    Write the test first
    Try and run the test
    Write the minimal amount of code for the test to run and check the failing test output
    Write enough code to make it pass
    Refactor
    Write the test first
    Try and run the test

Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Refactor
Write the test first
Try and run the test
Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Refactor
Write the test first
Try and run the test
Write enough code to make it pass
Refactor
Wrapping up
Structs, methods & interfaces
Write the test first
Try to run the test
Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Refactor
Write the test first
Try to run the test
Write the minimal amount of code for the test to run and check the failing test output
What are methods?
Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Refactor
Wait, what?
Decoupling
Further refactoring
Write the test first
Try to run the test
Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Refactor
Make sure your test output is helpful
Wrapping up
Maps
Write the test first
Try to run the test
Write the minimal amount of code for the test to run and check the output
Write enough code to make it pass
Refactor
Using a custom type
Write the test first
Try and run the test
Write the minimal amount of code for the test to run and check the output
Write enough code to make it pass
Refactor
Write the test first
Write the minimal amount of code for the test to run and check output
Write enough code to make it pass
Pointers, copies, et al
Refactor
Write the test first
Try to run test

Write the minimal amount of code for the test to run and check the output
Write enough code to make it pass
Refactor
Write the test first
Try and run the test
Write minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Write the test first
Try and run the test
Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Note on declaring a new error for Update
Write the test first
Try to run the test
Write the minimal amount of code for the test to run and check the failing test output
Write enough code to make it pass
Refactor
Try to run test
Write enough code to make it pass
Wrapping up

Learn Go with Tests -- Go Fundamentals (Excerpt)

 Excerpt assembled from github.com/quii/learn-go-with-tests, covering Go language fundamentals for cross-source comparison.

Hello, World

You can find all the code for this chapter here

It is traditional for your first program in a new language to be Hello, World.

- Create a folder wherever you like
- Put a new file in it called `hello.go` and put the following code inside it

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello, world")
}
```

To run it, type `go run hello.go`.

How it works

When you write a program in Go, you will have a `main` package defined with a `main` func inside it. Packages are ways of grouping up related Go code together.

The `func` keyword defines a function with a name and a body.

With `import "fmt"` we are importing a package which contains the `Println` function that we use to print.

How to test

How do you test this? It is good to separate your "domain" code from the outside world (side-effects). The `fmt.Println` is a side effect (printing to stdout), and the string we send in is our domain.

So let's separate these concerns so it's easier to test

```
package main

import "fmt"

func Hello() string {
    return "Hello, world"
}

func main() {
    fmt.Println(Hello())
}
```

We have created a new function with `func`, but this time, we've added another keyword, `string`, to the definition. This means this function returns a `string`.

Now create a new file called `hello_test.go` where we are going to write a test for our `Hello` function

```
package main

import "testing"

func TestHello(t *testing.T) {
    got := Hello()
    want := "Hello, world"

    if got != want {
        t.Errorf("got %q want %q", got, want)
    }
}
```

Go modules?

The next step is to run the tests. Enter `go test` in your terminal. If the tests pass, then you are probably using an earlier version of Go. However, if you are using Go 1.16 or later, the tests will likely not run. Instead, you will see an error message like this in the terminal:

```
$ go test
go: cannot find main module; see 'go help modules'
```

What's the problem? In a word, modules. Luckily, the problem is easy to fix. Enter `go mod init example.com/hello` in your terminal. That will create a new file with the following contents:

```
module example.com/hello

go 1.16
```

This file tells the `go` tools essential information about your code. If you planned to distribute your application, you would include where the code was available for download as well as information about dependencies. The name of the module, example.com/hello, usually refers to a URL where the module can be found and downloaded. For compatibility with tools we'll start using soon, make sure your module's name has a dot somewhere in it, like the dot in .com of example.com/hello. For now, your module file is minimal, and you can leave it that way. To read more about modules, you can check out the reference in the Golang documentation. We can get back to testing and learning Go now since the tests should run, even on Go 1.16.

In future chapters, you will need to run `go mod init SOMENAME` in each new folder before running commands like `go test` or `go build`.

Back to Testing

Run `go test` in your terminal. It should've passed! Just to check, try deliberately breaking the test by changing the `want` string.

Notice how you have not had to pick between multiple testing frameworks and then figure out how to install them. Everything you need is built into the language, and the syntax is the same as the rest of the code you will write.

Writing tests

Writing a test is just like writing a function, with a few rules

- It needs to be in a file with a name like `xxx_test.go`
- The test function must start with the word `Test`
- The test function takes one argument only `t *testing.T`
- To use the `*testing.T` type, you need to `import "testing"`, like we did with `fmt` in the other file

For now, it's enough to know that your `t` of type `*testing.T` is your "hook" into the testing framework so you can do things like `t.Fail()` when you want to fail.

We've covered some new topics:

`if`

If statements in Go are very much like other programming languages.

Declaring variables

We're declaring some variables with the syntax `varName := value`, which lets us reuse some values in our test for readability.

`t.Errorf`

We are calling the `Errorf` *method* on our `t`, which will print out a message and fail the test. The `f` stands for format, which allows us to build a string with values inserted into the placeholder values `%q`. When you make the test fail, it should be clear how it works.

You can read more about the placeholder strings in the fmt documentation. For tests, `%q` is very useful as it wraps your values in double quotes.

We will later explore the difference between methods and functions.

Go's documentation

Another quality-of-life feature of Go is the documentation. We just saw the documentation for the fmt package at the official package viewing website, and Go also provides ways for quickly getting at the documentation offline.

Go has a built-in tool, doc, which lets you examine any package installed on your system, or the module you're currently working on. To view that same documentation for the Printing verbs:

```
$ go doc fmt
package fmt // import "fmt"

Package fmt implements formatted I/O with functions analogous to C's
printf and
scanf. The format 'verbs' are derived from C's but are simpler.

# Printing

The verbs:

General:

    %v  the value in a default format
        when printing structs, the plus flag (%+v) adds field names
    %#v a Go-syntax representation of the value
    %T  a Go-syntax representation of the type of the value
    %%  a literal percent sign; consumes no value
...
```

Go's second tool for viewing documentation is the pkgsite command, which powers Go's official package viewing website. You can install pkgsite with `go install golang.org/x/pkgsite/cmd/pkgsite@latest`, then run it with `pkgsite -open .`. Go's install command will download the source files from that repository and build them into an executable binary. For a default installation of Go, that executable will be in `$HOME/go/bin` for Linux and macOS, and `%USERPROFILE%\go\bin` for Windows. If you have not already added those paths to your $PATH var, you might want to do so to make running go-installed commands easier.

The vast majority of the standard library has excellent documentation with examples. Navigating to http://localhost:8080/testing would be worthwhile to see what's available to you.

Hello, YOU

Now that we have a test, we can iterate on our software safely.

In the last example, we wrote the test *after* the code had been written so that you could get an example of how to write a test and declare a function. From this point on, we will be *writing tests first*.

Our next requirement is to let us specify the recipient of the greeting.

Let's start by capturing these requirements in a test. This is basic test-driven development and allows us to make sure our test is *actually* testing what we want. When you retrospectively write tests, there is the risk that your test may continue to pass even if the code doesn't work as intended.

```
        package main
```

```
import "testing"

func TestHello(t *testing.T) {
    got := Hello("Chris")
    want := "Hello, Chris"

    if got != want {
        t.Errorf("got %q want %q", got, want)
    }
}
```

Now run `go test`, you should have a compilation error

```
./hello_test.go:6:18: too many arguments in call to Hello
    have (string)
    want ()
```

When using a statically typed language like Go it is important to *listen to the compiler*. The compiler understands how your code should snap together and work so you don't have to.

In this case the compiler is telling you what you need to do to continue. We have to change our function `Hello` to accept an argument.

Edit the `Hello` function to accept an argument of type string

```
func Hello(name string) string {
    return "Hello, world"
}
```

If you try and run your tests again your `hello.go` will fail to compile because you're not passing an argument. Send in "world" to make it compile.

```
func main() {
    fmt.Println(Hello("world"))
}
```

Now when you run your tests, you should see something like

```
hello_test.go:10: got 'Hello, world' want 'Hello, Chris''
```

We finally have a compiling program but it is not meeting our requirements according to the test.

Let's make the test pass by using the name argument and concatenate it with `Hello,`

```
func Hello(name string) string {
    return "Hello, " + name
}
```

When you run the tests, they should now pass. Normally, as part of the TDD cycle, we should now *refactor*.

A note on source control

At this point, if you are using source control (which you should!) I would `commit` the code as it is. We have working software backed by a test.

I *wouldn't* push to main though, because I plan to refactor next. It is nice to commit at this point in case you somehow get into a mess with refactoring - you can always go back to the working version.

There's not a lot to refactor here, but we can introduce another language feature, *constants*.

Constants

Constants are defined like so

```
const englishHelloPrefix = "Hello, "
```

We can now refactor our code

```
const englishHelloPrefix = "Hello, "

func Hello(name string) string {
    return englishHelloPrefix + name
}
```

After refactoring, re-run your tests to make sure you haven't broken anything.

It's worth thinking about creating constants to capture the meaning of values and sometimes to aid performance.

Hello, world... again

The next requirement is when our function is called with an empty string it defaults to printing "Hello, World", rather than "Hello, ".

Start by writing a new failing test

```
func TestHello(t *testing.T) {
    t.Run("saying hello to people", func(t *testing.T) {
        got := Hello("Chris")
        want := "Hello, Chris"

        if got != want {
            t.Errorf("got %q want %q", got, want)
        }
    })
    t.Run("say 'Hello, World' when an empty string is supplied",
func(t *testing.T) {
        got := Hello("")
        want := "Hello, World"

        if got != want {
            t.Errorf("got %q want %q", got, want)
        }
    })
}
```

Here, we are introducing another tool in our testing arsenal: subtests. Sometimes, it is useful to group tests around a "thing" and then have subtests describing different scenarios.

A benefit of this approach is you can set up shared code that can be used in the other tests.

While we have a failing test, let's fix the code, using an `if`.

```
const englishHelloPrefix = "Hello, "

func Hello(name string) string {
    if name == "" {
```

```
            name = "World"
        }
        return englishHelloPrefix + name
    }
```

If we run our tests we should see it satisfies the new requirement and we haven't accidentally broken the other functionality.

It is important that your tests *are clear specifications* of what the code needs to do. But there is repeated code when we check if the message is what we expect.

Refactoring is not *just* for the production code!

Now that the tests are passing, we can and should refactor our tests.

```
    func TestHello(t *testing.T) {
        t.Run("saying hello to people", func(t *testing.T) {
            got := Hello("Chris")
            want := "Hello, Chris"
            assertCorrectMessage(t, got, want)
        })

        t.Run("empty string defaults to 'world'", func(t *testing.T) {
            got := Hello("")
            want := "Hello, World"
            assertCorrectMessage(t, got, want)
        })

    }

    func assertCorrectMessage(t testing.TB, got, want string) {
        t.Helper()
        if got != want {
            t.Errorf("got %q want %q", got, want)
        }
    }
```

What have we done here?

We've refactored our assertion into a new function. This reduces duplication and improves the readability of our tests. We need to pass in `t *testing.T` so that we can tell the test code to fail when we need to.

For helper functions, it's a good idea to accept a `testing.TB` which is an interface that `*testing.T` and `*testing.B` both satisfy, so you can call helper functions from a test, or a benchmark (don't worry if words like "interface" mean nothing to you right now, it will be covered later).

`t.Helper()` is needed to tell the test suite that this method is a helper. By doing this, when it fails, the line number reported will be in our *function call* rather than inside our test helper. This will help other developers track down problems more easily. If you still don't understand, comment it out, make a test fail and observe the test output. Comments in Go are a great way to add additional information to your code, or in this case, a quick way to tell the compiler to ignore a line. You can comment out the `t.Helper()` code by adding two forward slashes `//` at the beginning of the line. You should see that line turn grey or change to another color than the rest of your code to indicate it's now commented out.

When you have more than one argument of the same type (in our case two strings) rather than having `(got string, want string)` you can shorten it to `(got, want string)`.

Back to source control

Now that we are happy with the code, I would amend the previous commit so that we only check in the lovely version of our code with its test.

Discipline

Let's go over the cycle again

- Write a test
- Make the compiler pass
- Run the test, see that it fails and check the error message is meaningful
- Write enough code to make the test pass
- Refactor

On the face of it this may seem tedious but sticking to the feedback loop is important.

Not only does it ensure that you have *relevant tests*, it helps ensure *you design good software* by refactoring with the safety of tests.

Seeing the test fail is an important check because it also lets you see what the error message looks like. As a developer it can be very hard to work with a codebase when failing tests do not give a clear idea as to what the problem is.

By ensuring your tests are *fast* and setting up your tools so that running tests is simple you can get in to a state of flow when writing your code.

By not writing tests, you are committing to manually checking your code by running your software, which breaks your state of flow. You won't be saving yourself any time, especially in the long run.

Keep going! More requirements

Goodness me, we have more requirements. We now need to support a second parameter, specifying the language of the greeting. If a language is passed in that we do not recognise, just default to English.

We should be confident that we can easily use TDD to flesh out this functionality!

Write a test for a user passing in Spanish. Add it to the existing suite.

```
t.Run("in Spanish", func(t *testing.T) {
    got := Hello("Elodie", "Spanish")
    want := "Hola, Elodie"
    assertCorrectMessage(t, got, want)
})
```

Remember not to cheat! *Test first*. When you try to run the test, the compiler *should* complain because you are calling `Hello` with two arguments rather than one.

```
./hello_test.go:27:19: too many arguments in call to Hello
    have (string, string)
    want (string)
```

Fix the compilation problems by adding another string argument to `Hello`

```
func Hello(name string, language string) string {
    if name == "" {
        name = "World"
    }
    return englishHelloPrefix + name
}
```

When you try and run the test again it will complain about not passing through enough arguments to `Hello` in your other tests and in `hello.go`

```
./hello.go:15:19: not enough arguments in call to Hello
    have (string)
    want (string, string)
```

Fix them by passing through empty strings. Now all your tests should compile *and* pass, apart from our new scenario

```
hello_test.go:29: got 'Hello, Elodie' want 'Hola, Elodie'
```

We can use `if` here to check the language is equal to "Spanish" and if so change the message

```
func Hello(name string, language string) string {
    if name == "" {
        name = "World"
    }

    if language == "Spanish" {
        return "Hola, " + name
    }
    return englishHelloPrefix + name
}
```

The tests should now pass.

Now it is time to *refactor*. You should see some problems in the code, "magic" strings, some of which are repeated. Try and refactor it yourself, with every change make sure you re-run the tests to make sure your refactoring isn't breaking anything.

```
const spanish = "Spanish"
const englishHelloPrefix = "Hello, "
const spanishHelloPrefix = "Hola, "

func Hello(name string, language string) string {
    if name == "" {
        name = "World"
    }

    if language == spanish {
        return spanishHelloPrefix + name
    }
    return englishHelloPrefix + name
}
```

French

- Write a test asserting that if you pass in `"French"` you get `"Bonjour, "`
- See it fail, check the error message is easy to read
- Do the smallest reasonable change in the code

You may have written something that looks roughly like this

```
func Hello(name string, language string) string {
```

```
    if name == "" {
        name = "World"
    }

    if language == spanish {
        return spanishHelloPrefix + name
    }
    if language == french {
        return frenchHelloPrefix + name
    }
    return englishHelloPrefix + name
}
```

`switch`

When you have lots of `if` statements checking a particular value it is common to use a `switch` statement instead. We can use `switch` to refactor the code to make it easier to read and more extensible if we wish to add more language support later

```
func Hello(name string, language string) string {
    if name == "" {
        name = "World"
    }

    prefix := englishHelloPrefix

    switch language {
    case spanish:
        prefix = spanishHelloPrefix
    case french:
        prefix = frenchHelloPrefix
    }

    return prefix + name
}
```

Write a test to now include a greeting in the language of your choice and you should see how simple it is to extend our *amazing* function.

one...last...refactor?

You could argue that maybe our function is getting a little big. The simplest refactor for this would be to extract out some functionality into another function.

```
const (
    spanish = "Spanish"
    french  = "French"

    englishHelloPrefix = "Hello, "
    spanishHelloPrefix = "Hola, "
    frenchHelloPrefix  = "Bonjour, "
)

func Hello(name string, language string) string {
    if name == "" {
        name = "World"
    }

    return greetingPrefix(language) + name
```

```
}

func greetingPrefix(language string) (prefix string) {
    switch language {
    case french:
        prefix = frenchHelloPrefix
    case spanish:
        prefix = spanishHelloPrefix
    default:
        prefix = englishHelloPrefix
    }
    return
}
```

A few new concepts:

- In our function signature we have made a *named return value* (`prefix string`).
- This will create a variable called `prefix` in your function.
  - It will be assigned the "zero" value. This depends on the type, for example `int`s are 0 and for `string`s it is `""`.
    - You can return whatever it's set to by just calling `return` rather than `return prefix`.
  - This will display in the Go Doc for your function so it can make the intent of your code clearer.
- `default` in the switch case will be branched to if none of the other `case` statements match.
- The function name starts with a lowercase letter. In Go, public functions start with a capital letter, and private ones start with a lowercase letter. We don't want the internals of our algorithm exposed to the world, so we made this function private.
- Also, we can group constants in a block instead of declaring them on their own line. For readability, it's a good idea to use a line between sets of related constants.

Wrapping up

Who knew you could get so much out of `Hello, world`?

By now you should have some understanding of:

Some of Go's syntax around

- Writing tests
- Declaring functions, with arguments and return types
- `if`, `const` and `switch`
- Declaring variables and constants

The TDD process and *why* the steps are important

- *Write a failing test and see it fail* so we know we have written a *relevant* test for our requirements and seen that it produces an *easy to understand description of the failure*
- Writing the smallest amount of code to make it pass so we know we have working software
- *Then* refactor, backed with the safety of our tests to ensure we have well-crafted code that is easy to work with

In our case, we've gone from `Hello()` to `Hello("name")` and then to `Hello("name", "French")` in small, easy-to-understand steps.

Of course, this is trivial compared to "real-world" software, but the principles still stand. TDD is a skill that needs practice to develop, but by breaking problems down into smaller components that you can test, you will have a much easier time writing software.

Integers

You can find all the code for this chapter here

Integers work as you would expect. Let's write an `Add` function to try things out. Create a test file called `adder_test.go` and write this code.

Note: Go source files can only have one `package` per directory. Make sure that your files are organised into their own packages. Here is a good explanation on this.

Your project directory might look something like this:

```
learnGoWithTests
    |
    |-> helloworld
    |    |- hello.go
    |    |- hello_test.go
    |
    |-> integers
    |    |- adder_test.go
    |
    |- go.mod
    |- README.md
```

Write the test first

```go
package integers

import "testing"

func TestAdder(t *testing.T) {
    sum := Add(2, 2)
    expected := 4

    if sum != expected {
        t.Errorf("expected '%d' but got '%d'", expected, sum)
    }
}
```

You will notice that we're using `%d` as our format strings rather than `%q`. That's because we want it to print an integer rather than a string.

Also note that we are no longer using the main package, instead we've defined a package named `integers`, as the name suggests this will group functions for working with integers such as `Add`.

Try and run the test

Run the test `go test`

Inspect the compilation error

```
./adder_test.go:6:9: undefined: Add
```

Write the minimal amount of code for the test to run and check the failing test output

Write enough code to satisfy the compiler *and that's all* - remember we want to check that our tests fail for the correct reason.

```
package integers

func Add(x, y int) int {
    return 0
}
```

Remember, when you have more than one argument of the same type (in our case two integers) rather than having `(x int, y int)` you can shorten it to `(x, y int)`.

Now run the tests, and we should be happy that the test is correctly reporting what is wrong.

```
adder_test.go:10: expected '4' but got '0'
```

If you have noticed we learnt about *named return value* in the last section but aren't using the same here. It should generally be used when the meaning of the result isn't clear from context, in our case it's pretty much clear that `Add` function will add the parameters. You can refer this wiki for more details.

Write enough code to make it pass

In the strictest sense of TDD we should now write the *minimal amount of code to make the test pass*. A pedantic programmer may do this

```
func Add(x, y int) int {
    return 4
}
```

Ah hah! Foiled again, TDD is a sham right?

We could write another test, with some different numbers to force that test to fail but that feels like a game of cat and mouse.

Once we're more familiar with Go's syntax I will introduce a technique called "*Property Based Testing*", which would stop annoying developers and help you find bugs.

For now, let's fix it properly

```
func Add(x, y int) int {
    return x + y
}
```

If you re-run the tests they should pass.

Refactor

There's not a lot in the *actual* code we can really improve on here.

We explored earlier how by naming the return argument it appears in the documentation but also in most developer's text editors.

This is great because it aids the usability of code you are writing. It is preferable that a user can understand the usage of your code by just looking at the type signature and documentation.

You can add documentation to functions with comments, and these will appear in Go Doc just like when you look at the standard library's documentation.

```
// Add takes two integers and returns the sum of them.
func Add(x, y int) int {
    return x + y
}
```

Testable Examples

If you really want to go the extra mile you can make Testable Examples. You will find many examples in the standard library documentation.

Often code examples that can be found outside the codebase, such as a readme file, become out of date and incorrect compared to the actual code because they don't get checked.

Example functions are compiled whenever tests are executed. Because such examples are validated by the Go compiler, you can be confident your documentation's examples always reflect current code behavior.

Example functions begin with `Example` (much like test functions begin with `Test`), and reside in a package's `_test.go` files. Add the following `ExampleAdd` function to the `adder_test.go` file.

```
func ExampleAdd() {
    sum := Add(1, 5)
    fmt.Println(sum)
    // Output: 6
}
```

(If your editor doesn't automatically import packages for you, the compilation step will fail because you will be missing `import "fmt"` in `adder_test.go`. It is strongly recommended you research how to have these kind of errors fixed for you automatically in whatever editor you are using.)

Adding this code will cause the example to appear in your documentation, making your code even more accessible. If ever your code changes so that the example is no longer valid, your build will fail.

Running the package's test suite, we can see the example `ExampleAdd` function is executed with no further arrangement from us:

```
$ go test -v
=== RUN   TestAdder
--- PASS: TestAdder (0.00s)
=== RUN   ExampleAdd
--- PASS: ExampleAdd (0.00s)
```

Notice the special format of the comment, `// Output: 6`. While the example will always be compiled, adding this comment means the example will also be executed. Go ahead and temporarily remove the comment `// Output: 6`, then run `go test`, and you will see `ExampleAdd` is no longer executed.

Examples without output comments are useful for demonstrating code that cannot run as unit tests, such as that which accesses the network, while guaranteeing the example at least compiles.

To view example documentation, let's take a quick look at `pkgsite`. Before navigating to your project's directory, make sure you have installed `pkgsite` by running the following command: `go install golang.org/x/pkgsite/cmd/pkgsite@latest`, then run `pkgsite -open .`, which should open a web browser for you, pointing to `http://localhost:8080`. Inside here you'll see a list of all of Go's Standard Library packages, plus Third Party packages you have installed, under which you should see your example documentation for `github.com/quii/learn-go-with-tests`. Follow that link, and then look under `Integers`, then under `func Add`, then expand `Example` and you should see the example you added for `sum := Add(1, 5)`.

If you publish your code with examples to a public URL, you can share the documentation of your code at pkg.go.dev. For example, here is the finalised API for this chapter. This web interface allows you to search for documentation of standard library packages and third-party packages.

Wrapping up

What we have covered:

- More practice of the TDD workflow
- Integers, addition
- Writing better documentation so users of our code can understand its usage quickly
- Examples of how to use our code, which are checked as part of our tests

Iteration

You can find all the code for this chapter here

To do stuff repeatedly in Go, you'll need `for`. In Go there are no `while`, `do`, `until` keywords, you can only use `for`. Which is a good thing!

Let's write a test for a function that repeats a character 5 times.

There's nothing new so far, so try and write it yourself for practice.

Write the test first

```
package iteration

import "testing"

func TestRepeat(t *testing.T) {
    repeated := Repeat("a")
    expected := "aaaaa"

    if repeated != expected {
        t.Errorf("expected %q but got %q", expected, repeated)
    }
}
```

Try and run the test

```
./repeat_test.go:6:14: undefined: Repeat
```

Write the minimal amount of code for the test to run and check the failing test output

Keep the discipline! You don't need to know anything new right now to make the test fail properly.

All you need to do right now is enough to make it compile so you can check your test is written well.

```
package iteration

func Repeat(character string) string {
    return ""
}
```

Isn't it nice to know you already know enough Go to write tests for some basic problems? This means you can now play with the production code as much as you like and know it's behaving as you'd hope.

```
repeat_test.go:10: expected 'aaaaa' but got ''
```

Write enough code to make it pass

The `for` syntax is very unremarkable and follows most C-like languages.

```
func Repeat(character string) string {
    var repeated string
    for i := 0; i < 5; i++ {
        repeated = repeated + character
    }
    return repeated
}
```

Unlike other languages like C, Java, or JavaScript there are no parentheses surrounding the three components of the for statement and the braces `{ }` are always required. You might wonder what is happening in the row

```
    var repeated string
```

as we've been using `:=` so far to declare and initializing variables. However, `:=` is simply short hand for both steps. Here we are declaring a `string` variable only. Hence, the explicit version. We can also use `var` to declare functions, as we'll see later on.

Run the test and it should pass.

Additional variants of the for loop are described here.

Refactor

Now it's time to refactor and introduce another construct `+=` assignment operator.

```
const repeatCount = 5

func Repeat(character string) string {
    var repeated string
    for i := 0; i < repeatCount; i++ {
        repeated += character
    }
    return repeated
}
```

+= called "*the Add AND assignment operator*", adds the right operand to the left operand and assigns the result to left operand. It works with other types like integers.

Benchmarking

Writing benchmarks in Go is another first-class feature of the language and it is very similar to writing tests.

```
func BenchmarkRepeat(b *testing.B) {
    for b.Loop() {
        Repeat("a")
    }
}
```

You'll see the code is very similar to a test.

The `testing.B` gives you access to the loop function. `Loop()` returns true as long as the benchmark should continue running.

When the benchmark code is executed, it measures how long it takes. After `Loop()` returns false, `b.N` contains the total number of iterations that ran.

The number of times the code is run shouldn't matter to you, the framework will determine what is a "good" value for that to let you have some decent results.

To run the benchmarks do `go test -bench=.` (or if you're in Windows Powershell `go test -bench="."`)

```
goos: darwin
goarch: amd64
pkg: github.com/quii/learn-go-with-tests/for/v4
10000000           136 ns/op
PASS
```

What `136 ns/op` means is our function takes on average 136 nanoseconds to run (on my computer). Which is pretty ok! To test this it ran it 10000000 times.

Note: By default benchmarks are run sequentially.

Only the body of the loop is timed; it automatically excludes setup and cleanup code from benchmark timing. A typical benchmark is structured like:

```
func Benchmark(b *testing.B) {
    //... setup ...
    for b.Loop() {
        //... code to measure ...
    }
    //... cleanup ...
}
```

Strings in Go are immutable, meaning every concatenation, such as in our `Repeat` function, involves copying memory to accommodate the new string. This impacts performance, particularly during heavy string concatenation.

The standard library provides the `strings.Builder`stringsBuilder type which minimizes memory copying. It implements a `WriteString` method which we can use to concatenate strings:

```
const repeatCount = 5

func Repeat(character string) string {
    var repeated strings.Builder
    for i := 0; i < repeatCount; i++ {
        repeated.WriteString(character)
    }
    return repeated.String()
}
```

Note: We have to call the `String` method to retrieve the final result.

We can use `BenchmarkRepeat` to confirm that `strings.Builder` significantly improves performance. Run `go test -bench=. -benchmem`:

```
goos: darwin
goarch: amd64
pkg: github.com/quii/learn-go-with-tests/for/v4
10000000           25.70 ns/op           8 B/op           1
allocs/op
PASS
```

The `-benchmem` flag reports information about memory allocations:

- `B/op`: the number of bytes allocated per iteration
- `allocs/op`: the number of memory allocations per iteration

Practice exercises

- Change the test so a caller can specify how many times the character is repeated and then fix the code
- Write `ExampleRepeat` to document your function
- Have a look through the strings package. Find functions you think could be useful and experiment with them by writing tests like we have here. Investing time learning the standard library will really pay off over time.

Wrapping up

- More TDD practice
- Learned `for`
- Learned how to write benchmarks

Arrays and slices

You can find all the code for this chapter here

Arrays allow you to store multiple elements of the same type in a variable in a particular order.

When you have arrays, it is very common to have to iterate over them. So let's use our new-found knowledge of `for` to make a `Sum` function. `Sum` will take an array of numbers and return the total.

Let's use our TDD skills

Write the test first

Create a new folder to work in. Create a new file called `sum_test.go` and insert the following:

```
package main

import "testing"

func TestSum(t *testing.T) {

    numbers := [5]int{1, 2, 3, 4, 5}

    got := Sum(numbers)
    want := 15

    if got != want {
        t.Errorf("got %d want %d given, %v", got, want, numbers)
    }
}
```

Arrays have a *fixed capacity* which you define when you declare the variable. We can initialize an array in two ways:

- [N]type{value1, value2, ..., valueN} e.g. `numbers := [5]int{1, 2, 3, 4, 5}`
- [...]type{value1, value2, ..., valueN} e.g. `numbers := [...]int{1, 2, 3, 4, 5}`

It is sometimes useful to also print the inputs to the function in the error message. Here, we are using the `%v` placeholder to print the "default" format, which works well for arrays.

Read more about the format strings

Try to run the test

If you had initialized go mod with `go mod init main` you will be presented with an error `_testmain.go:13:2: cannot import "main"`. This is because according to common practice, package main will only contain integration of other packages and not unit-testable code and hence Go will not allow you to import a package with name `main`.

To fix this, you can rename the main module in `go.mod` to any other name.

Once the above error is fixed, if you run `go test` the compiler will fail with the familiar `./sum_test.go:10:15: undefined: Sum` error. Now we can proceed with writing the actual method to be tested.

Write the minimal amount of code for the test to run and check the failing test output

In `sum.go`

```
package main

func Sum(numbers [5]int) int {
    return 0
}
```

Your test should now fail with *a clear error message*

```
sum_test.go:13: got 0 want 15 given, [1 2 3 4 5]
```

Write enough code to make it pass

```
func Sum(numbers [5]int) int {
    sum := 0
    for i := 0; i < 5; i++ {
        sum += numbers[i]
    }
    return sum
}
```

To get the value out of an array at a particular index, just use `array[index]` syntax. In this case, we are using `for` to iterate 5 times to work through the array and add each item onto `sum`.

Refactor

Let's introduce `range` to help clean up our code

```
func Sum(numbers [5]int) int {
    sum := 0
    for _, number := range numbers {
        sum += number
    }
    return sum
}
```

`range` lets you iterate over an array. On each iteration, `range` returns two values - the index and the value. We are choosing to ignore the index value by using `_` blank identifier.

Arrays and their type

An interesting property of arrays is that the size is encoded in its type. If you try to pass an `[4]int` into a function that expects `[5]int`, it won't compile. They are different types so it's just the same as trying to pass a `string` into a function that wants an `int`.

You may be thinking it's quite cumbersome that arrays have a fixed length, and most of the time you probably won't be using them!

Go has *slices* which do not encode the size of the collection and instead can have any size.

The next requirement will be to sum collections of varying sizes.

Write the test first

We will now use the slice type which allows us to have collections of any size. The syntax is very similar to arrays, you just omit the size when declaring them

`mySlice := []int{1,2,3}` rather than `myArray := [3]int{1,2,3}`

```
func TestSum(t *testing.T) {

    t.Run("collection of 5 numbers", func(t *testing.T) {
        numbers := [5]int{1, 2, 3, 4, 5}

        got := Sum(numbers)
        want := 15

        if got != want {
            t.Errorf("got %d want %d given, %v", got, want, numbers)
        }
    })

    t.Run("collection of any size", func(t *testing.T) {
        numbers := []int{1, 2, 3}

        got := Sum(numbers)
        want := 6

        if got != want {
            t.Errorf("got %d want %d given, %v", got, want, numbers)
        }
    })

}
```

Try and run the test

This does not compile

```
./sum_test.go:22:13: cannot use numbers (type []int) as type [5]int
in argument to Sum
```

Write the minimal amount of code for the test to run and check the failing test output

The problem here is we can either

- Break the existing API by changing the argument to `Sum` to be a slice rather than an array. When we do this, we will potentially ruin someone's day because our *other* test will no longer compile!
- Create a new function

In our case, no one else is using our function, so rather than having two functions to maintain, let's have just one.

```
func Sum(numbers []int) int {
    sum := 0
    for _, number := range numbers {
        sum += number
    }
    return sum
}
```

If you try to run the tests they will still not compile, you will have to change the first test to pass in a slice rather than an array.

Write enough code to make it pass

It turns out that fixing the compiler problems were all we need to do here and the tests pass!

Refactor

We already refactored `Sum` - all we did was replace arrays with slices, so no extra changes are required. Remember that we must not neglect our test code in the refactoring stage - we can further improve our `Sum` tests.

```
        func TestSum(t *testing.T) {

            t.Run("collection of 5 numbers", func(t *testing.T) {
                numbers := []int{1, 2, 3, 4, 5}

                got := Sum(numbers)
                want := 15

                if got != want {
                    t.Errorf("got %d want %d given, %v", got, want, numbers)
                }
            })

            t.Run("collection of any size", func(t *testing.T) {
                numbers := []int{1, 2, 3}

                got := Sum(numbers)
                want := 6

                if got != want {
                    t.Errorf("got %d want %d given, %v", got, want, numbers)
                }
            })

        }
```

It is important to question the value of your tests. It should not be a goal to have as many tests as possible, but rather to have as much *confidence* as possible in your code base. Having too many tests can turn in to a real problem and it just adds more overhead in maintenance. **Every test has a cost**.

In our case, you can see that having two tests for this function is redundant. If it works for a slice of one size it's very likely it'll work for a slice of any size (within reason).

Go's built-in testing toolkit features a coverage tool. Whilst striving for 100% coverage should not be your end goal, the coverage tool can help identify areas of your code not covered by tests. If you have been strict with TDD, it's quite likely you'll have close to 100% coverage anyway.

Try running

```
go test -cover
```

You should see

```
PASS
coverage: 100.0% of statements
```

Now delete one of the tests and check the coverage again.

Now that we are happy we have a well-tested function you should commit your great work before taking on the next challenge.

We need a new function called `SumAll` which will take a varying number of slices, returning a new slice containing the totals for each slice passed in.

For example

`SumAll([]int{1,2}, []int{0,9})` would return `[]int{3, 9}`

or

`SumAll([]int{1,1,1})` would return `[]int{3}`

Write the test first

```
func TestSumAll(t *testing.T) {

    got := SumAll([]int{1, 2}, []int{0, 9})
    want := []int{3, 9}

    if got != want {
        t.Errorf("got %v want %v", got, want)
    }
}
```

Try and run the test

```
./sum_test.go:23:9: undefined: SumAll
```

Write the minimal amount of code for the test to run and check the failing test output

We need to define `SumAll` according to what our test wants.

Go can let you write *variadic functions* that can take a variable number of arguments.

```
func SumAll(numbersToSum ...[]int) []int {
    return nil
}
```

This is valid, but our tests still won't compile!

```
./sum_test.go:26:9: invalid operation: got != want (slice can only
be compared to nil)
```

Go does not let you use equality operators with slices. You *could* write a function to iterate over each `got` and `want` slice and check their values, but what if we had a more convenient way to do this?

From Go 1.21, slices standard package is available, which has slices.Equal function to do a simple shallow compare on slices, where you don't need to worry about the types like the above case. Note that

this function expects the elements to be comparable. So, it can't be applied to slices with non-comparable elements like 2D slices.

Let's go ahead and put this into practice!

```
func TestSumAll(t *testing.T) {

    got := SumAll([]int{1, 2}, []int{0, 9})
    want := []int{3, 9}

    if !slices.Equal(got, want) {
        t.Errorf("got %v want %v", got, want)
    }
}
```

You should have test output like the following: `sum_test.go:30: got [] want [3 9]`

Write enough code to make it pass

What we need to do is iterate over the varargs, calculate the sum using our existing `Sum` function, then add it to the slice we will return

```
func SumAll(numbersToSum ...[]int) []int {
    lengthOfNumbers := len(numbersToSum)
    sums := make([]int, lengthOfNumbers)

    for i, numbers := range numbersToSum {
        sums[i] = Sum(numbers)
    }

    return sums
}
```

Lots of new things to learn!

There's a new way to create a slice. `make` allows you to create a slice with a starting capacity of the `len` of the `numbersToSum` we need to work through. The length of a slice is the number of elements it holds `len(mySlice)`, while the capacity is the number of elements it can hold in the underlying array `cap(mySlice)`, e.g., `make([]int, 0, 5)` creates a slice with length 0 and capacity 5.

You can index slices like arrays with `mySlice[N]` to get the value out or assign it a new value with `=`

The tests should now pass.

Refactor

As mentioned, slices have a capacity. If you have a slice with a capacity of 2 and try to do `mySlice[10] = 1` you will get a *runtime* error.

However, you can use the `append` function which takes a slice and a new value, then returns a new slice with all the items in it.

```
func SumAll(numbersToSum ...[]int) []int {
    var sums []int
    for _, numbers := range numbersToSum {
        sums = append(sums, Sum(numbers))
    }
```

```
    return sums
}
```

In this implementation, we are worrying less about capacity. We start with an empty slice `sums` and append to it the result of `Sum` as we work through the varargs.

Our next requirement is to change `SumAll` to `SumAllTails`, where it will calculate the totals of the "tails" of each slice. The tail of a collection is all items in the collection except the first one (the "head").

Write the test first

```
func TestSumAllTails(t *testing.T) {
    got := SumAllTails([]int{1, 2}, []int{0, 9})
    want := []int{2, 9}

    if !reflect.DeepEqual(got, want) {
        t.Errorf("got %v want %v", got, want)
    }
}
```

Try and run the test

```
./sum_test.go:26:9: undefined: SumAllTails
```

Write the minimal amount of code for the test to run and check the failing test output

Rename the function to `SumAllTails` and re-run the test

```
sum_test.go:30: got [3 9] want [2 9]
```

Write enough code to make it pass

```
func SumAllTails(numbersToSum ...[]int) []int {
    var sums []int
    for _, numbers := range numbersToSum {
        tail := numbers[1:]
        sums = append(sums, Sum(tail))
    }

    return sums
}
```

Slices can be sliced! The syntax is `slice[low:high]`. If you omit the value on one of the sides of the `:` it captures everything to that side of it. In our case, we are saying "take from 1 to the end" with `numbers[1:]`. You may wish to spend some time writing other tests around slices and experiment with the slice operator to get more familiar with it.

Refactor

Not a lot to refactor this time.

What do you think would happen if you passed in an empty slice into our function? What is the "tail" of an empty slice? What happens when you tell Go to capture all elements from `myEmptySlice[1:]`?

Write the test first

```
func TestSumAllTails(t *testing.T) {

    t.Run("make the sums of some slices", func(t *testing.T) {
        got := SumAllTails([]int{1, 2}, []int{0, 9})
        want := []int{2, 9}

        if !reflect.DeepEqual(got, want) {
            t.Errorf("got %v want %v", got, want)
        }
    })

    t.Run("safely sum empty slices", func(t *testing.T) {
        got := SumAllTails([]int{}, []int{3, 4, 5})
        want := []int{0, 9}

        if !reflect.DeepEqual(got, want) {
            t.Errorf("got %v want %v", got, want)
        }
    })

}
```

Try and run the test

```
panic: runtime error: slice bounds out of range [recovered]
    panic: runtime error: slice bounds out of range
```

Oh no! It's important to note that while the test *has compiled*, it *has a runtime error*.

Compile time errors are our friend because they help us write software that works, runtime errors are our enemies because they affect our users.

Write enough code to make it pass

```
func SumAllTails(numbersToSum ...[]int) []int {
    var sums []int
    for _, numbers := range numbersToSum {
        if len(numbers) == 0 {
            sums = append(sums, 0)
        } else {
            tail := numbers[1:]
            sums = append(sums, Sum(tail))
        }
    }

    return sums
}
```

Refactor

Our tests have some repeated code around the assertions again, so let's extract those into a function.

```
func TestSumAllTails(t *testing.T) {

    checkSums := func(t testing.TB, got, want []int) {
        t.Helper()
        if !reflect.DeepEqual(got, want) {
            t.Errorf("got %v want %v", got, want)
        }
    }

    t.Run("make the sums of tails of", func(t *testing.T) {
        got := SumAllTails([]int{1, 2}, []int{0, 9})
        want := []int{2, 9}
        checkSums(t, got, want)
    })

    t.Run("safely sum empty slices", func(t *testing.T) {
        got := SumAllTails([]int{}, []int{3, 4, 5})
        want := []int{0, 9}
        checkSums(t, got, want)
    })

}
```

We could've created a new function `checkSums` like we normally do, but in this case, we're showing a new technique, assigning a function to a variable. It might look strange but, it's no different to assigning a variable to a `string`, or an `int`, functions in effect are values too.

It's not shown here, but this technique can be useful when you want to bind a function to other local variables in "scope" (e.g between some `{}`). It also allows you to reduce the surface area of your API.

By defining this function inside the test, it cannot be used by other functions in this package. Hiding variables and functions that don't need to be exported is an important design consideration.

A handy side-effect of this is this adds a little type-safety to our code. If a developer mistakenly adds a new test with `checkSums(t, got, "dave")` the compiler will stop them in their tracks.

```
$ go test
./sum_test.go:52:21: cannot use "dave" (type string) as type []int
in argument to checkSums
```

Wrapping up

We have covered

- Arrays
- Slices
  - The various ways to make them
  - How they have a *fixed* capacity but you can create new slices from old ones using `append`
  - How to slice, slices!
- `len` to get the length of an array or slice
- Test coverage tool
- `reflect.DeepEqual` and why it's useful but can reduce the type-safety of your code

We've used slices and arrays with integers but they work with any other type too, including arrays/slices themselves. So you can declare a variable of `[][]string` if you need to.

Check out the Go blog post on slices for an in-depth look into slices. Try writing more tests to solidify what you learn from reading it.

Another handy way to experiment with Go other than writing tests is the Go playground. You can try most things out and you can easily share your code if you need to ask questions. I have made a go playground with a slice in it for you to experiment with.

Here is an example of slicing an array and how changing the slice affects the original array; but a "copy" of the slice will not affect the original array. Another example of why it's a good idea to make a copy of a slice after slicing a very large slice.

Structs, methods & interfaces

You can find all the code for this chapter here

Suppose that we need some geometry code to calculate the perimeter of a rectangle given a height and width. We can write a `Perimeter(width float64, height float64)` function, where `float64` is for floating-point numbers like `123.45`.

The TDD cycle should be pretty familiar to you by now.

Write the test first

```
func TestPerimeter(t *testing.T) {
    got := Perimeter(10.0, 10.0)
    want := 40.0

    if got != want {
        t.Errorf("got %.2f want %.2f", got, want)
    }
}
```

Notice the new format string? The `f` is for our `float64` and the `.2` means print 2 decimal places.

Try to run the test

```
./shapes_test.go:6:9: undefined: Perimeter
```

Write the minimal amount of code for the test to run and check the failing test output

```
func Perimeter(width float64, height float64) float64 {
    return 0
}
```

Results in `shapes_test.go:10: got 0.00 want 40.00`.

Write enough code to make it pass

```
func Perimeter(width float64, height float64) float64 {
    return 2 * (width + height)
}
```

So far, so easy. Now let's create a function called `Area(width, height float64)` which returns the area of a rectangle.

Try to do it yourself, following the TDD cycle.

You should end up with tests like this

```
func TestPerimeter(t *testing.T) {
    got := Perimeter(10.0, 10.0)
    want := 40.0

    if got != want {
        t.Errorf("got %.2f want %.2f", got, want)
    }
}

func TestArea(t *testing.T) {
    got := Area(12.0, 6.0)
    want := 72.0

    if got != want {
        t.Errorf("got %.2f want %.2f", got, want)
    }
}
```

And code like this

```
func Perimeter(width float64, height float64) float64 {
    return 2 * (width + height)
}

func Area(width float64, height float64) float64 {
    return width * height
}
```

Refactor

Our code does the job, but it doesn't contain anything explicit about rectangles. An unwary developer might try to supply the width and height of a triangle to these functions without realising they will return the wrong answer.

We could just give the functions more specific names like `RectangleArea`. A neater solution is to define our own *type* called `Rectangle` which encapsulates this concept for us.

We can create a simple type using a **struct**. A struct is just a named collection of fields where you can store data.

Declare a struct in your `shapes.go` file like this

```
type Rectangle struct {
    Width  float64
    Height float64
}
```

Now let's refactor the tests to use `Rectangle` instead of plain `float64`s.

```
func TestPerimeter(t *testing.T) {
    rectangle := Rectangle{10.0, 10.0}
    got := Perimeter(rectangle)
    want := 40.0

    if got != want {
        t.Errorf("got %.2f want %.2f", got, want)
    }
}

func TestArea(t *testing.T) {
    rectangle := Rectangle{12.0, 6.0}
    got := Area(rectangle)
    want := 72.0

    if got != want {
        t.Errorf("got %.2f want %.2f", got, want)
    }
}
```

Remember to run your tests before attempting to fix. The tests should show a helpful error like

```
./shapes_test.go:7:18: not enough arguments in call to Perimeter
    have (Rectangle)
    want (float64, float64)
```

You can access the fields of a struct with the syntax of `myStruct.field`.

Change the two functions to fix the test.

```
func Perimeter(rectangle Rectangle) float64 {
    return 2 * (rectangle.Width + rectangle.Height)
}

func Area(rectangle Rectangle) float64 {
    return rectangle.Width * rectangle.Height
}
```

I hope you'll agree that passing a `Rectangle` to a function conveys our intent more clearly, but there are more benefits of using structs that we will cover later.

Our next requirement is to write an `Area` function for circles.

Write the test first

```
func TestArea(t *testing.T) {

    t.Run("rectangles", func(t *testing.T) {
        rectangle := Rectangle{12, 6}
        got := Area(rectangle)
        want := 72.0

        if got != want {
            t.Errorf("got %g want %g", got, want)
        }
    })

    t.Run("circles", func(t *testing.T) {
        circle := Circle{10}
        got := Area(circle)
        want := 314.1592653589793
```

```
            if got != want {
                t.Errorf("got %g want %g", got, want)
            }
        })

    }
```

As you can see, the `f` has been replaced by `g`, with good reason. Use of `g` will print a more precise decimal number in the error message (fmt options). For example, using a radius of 1.5 in a circle area calculation, `f` would show `7.068583` whereas `g` would show `7.0685834705770345`.

Try to run the test

```
./shapes_test.go:28:13: undefined: Circle
```

Write the minimal amount of code for the test to run and check the failing test output

We need to define our `Circle` type.

```
type Circle struct {
    Radius float64
}
```

Now try to run the tests again

```
./shapes_test.go:29:14: cannot use circle (type Circle) as type
Rectangle in argument to Area
```

Some programming languages allow you to do something like this:

```
func Area(circle Circle) float64       {}
func Area(rectangle Rectangle) float64 {}
```

But you cannot in Go

```
./shapes.go:20:32: Area redeclared in this block
```

We have two choices:

- You can have functions with the same name declared in different *packages*. So we could create our `Area(Circle)` in a new package, but that feels overkill here.
- We can define *methods* on our newly defined types instead.

What are methods?

So far we have only been writing *functions* but we have been using some methods. When we call `t.Errorf` we are calling the method `Errorf` on the instance of our `t` (`testing.T`).

A method is a function with a receiver. A method declaration binds an identifier, the method name, to a method, and associates the method with the receiver's base type.

Methods are very similar to functions but they are called by invoking them on an instance of a particular type. Where you can just call functions wherever you like, such as `Area(rectangle)` you can only call methods on "things".

An example will help so let's change our tests first to call methods instead and then fix the code.

```
func TestArea(t *testing.T) {

    t.Run("rectangles", func(t *testing.T) {
        rectangle := Rectangle{12, 6}
        got := rectangle.Area()
        want := 72.0

        if got != want {
            t.Errorf("got %g want %g", got, want)
        }
    })

    t.Run("circles", func(t *testing.T) {
        circle := Circle{10}
        got := circle.Area()
        want := 314.1592653589793

        if got != want {
            t.Errorf("got %g want %g", got, want)
        }
    })

}
```

If we try to run the tests, we get

```
./shapes_test.go:19:19: rectangle.Area undefined (type Rectangle has
no field or method Area)
./shapes_test.go:29:16: circle.Area undefined (type Circle has no
field or method Area)
```

> type Circle has no field or method Area

I would like to reiterate how great the compiler is here. It is so important to take the time to slowly read the error messages you get, it will help you in the long run.

Write the minimal amount of code for the test to run and check the failing test output

Let's add some methods to our types

```
type Rectangle struct {
    Width  float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return 0
}

type Circle struct {
    Radius float64
```

```
}

func (c Circle) Area() float64 {
    return 0
}
```

The syntax for declaring methods is almost the same as functions and that's because they're so similar. The only difference is the syntax of the method receiver `func (receiverName ReceiverType) MethodName(args)`.

When your method is called on a variable of that type, you get your reference to its data via the `receiverName` variable. In many other programming languages this is done implicitly and you access the receiver via `this`.

It is a convention in Go to have the receiver variable be the first letter of the type.

```
r Rectangle
```

If you try to re-run the tests they should now compile and give you some failing output.

Write enough code to make it pass

Now let's make our rectangle tests pass by fixing our new method

```
func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}
```

If you re-run the tests the rectangle tests should be passing but circle should still be failing.

To make circle's `Area` function pass we will borrow the `Pi` constant from the `math` package (remember to import it).

```
func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}
```

Refactor

There is some duplication in our tests.

All we want to do is take a collection of *shapes*, call the `Area()` method on them and then check the result.

We want to be able to write some kind of `checkArea` function that we can pass both `Rectangles` and `Circles` to, but fail to compile if we try to pass in something that isn't a shape.

With Go, we can codify this intent with **interfaces**.

Interfaces are a very powerful concept in statically typed languages like Go because they allow you to make functions that can be used with different types and create highly-decoupled code whilst still maintaining type-safety.

Let's introduce this by refactoring our tests.

```
func TestArea(t *testing.T) {
```

```
        checkArea := func(t testing.TB, shape Shape, want float64) {
            t.Helper()
            got := shape.Area()
            if got != want {
                t.Errorf("got %g want %g", got, want)
            }
        }

        t.Run("rectangles", func(t *testing.T) {
            rectangle := Rectangle{12, 6}
            checkArea(t, rectangle, 72.0)
        })

        t.Run("circles", func(t *testing.T) {
            circle := Circle{10}
            checkArea(t, circle, 314.1592653589793)
        })

    }
```

We are creating a helper function like we have in other exercises but this time we are asking for a `Shape` to be passed in. If we try to call this with something that isn't a shape, then it will not compile.

How does something become a shape? We just tell Go what a `Shape` is using an interface declaration

```
type Shape interface {
    Area() float64
}
```

We're creating a new `type` just like we did with `Rectangle` and `Circle` but this time it is an `interface` rather than a `struct`.

Once you add this to the code, the tests will pass.

Wait, what?

This is quite different to interfaces in most other programming languages. Normally you have to write code to say `My type Foo implements interface Bar`.

But in our case

- `Rectangle` has a method called `Area` that returns a `float64` so it satisfies the `Shape` interface
- `Circle` has a method called `Area` that returns a `float64` so it satisfies the `Shape` interface
- `string` does not have such a method, so it doesn't satisfy the interface
- etc.

In Go **interface resolution is implicit**. If the type you pass in matches what the interface is asking for, it will compile.

Decoupling

Notice how our helper does not need to concern itself with whether the shape is a `Rectangle` or a `Circle` or a `Triangle`. By declaring an interface, the helper is *decoupled* from the concrete types and only has the method it needs to do its job.

This kind of approach of using interfaces to declare **only what you need** is very important in software design and will be covered in more detail in later sections.

Further refactoring

Now that you have some understanding of structs we can introduce "table driven tests".

Table driven tests are useful when you want to build a list of test cases that can be tested in the same manner.

```
func TestArea(t *testing.T) {

    areaTests := []struct {
        shape Shape
        want  float64
    }{
        {Rectangle{12, 6}, 72.0},
        {Circle{10}, 314.1592653589793},
    }

    for _, tt := range areaTests {
        got := tt.shape.Area()
        if got != tt.want {
            t.Errorf("got %g want %g", got, tt.want)
        }
    }

}
```

The only new syntax here is creating an "anonymous struct", `areaTests`. We are declaring a slice of structs by using `[]struct` with two fields, the `shape` and the `want`. Then we fill the slice with cases.

We then iterate over them just like we do any other slice, using the struct fields to run our tests.

You can see how it would be very easy for a developer to introduce a new shape, implement `Area` and then add it to the test cases. In addition, if a bug is found with `Area` it is very easy to add a new test case to exercise it before fixing it.

Table driven tests can be a great item in your toolbox, but be sure that you have a need for the extra noise in the tests. They are a great fit when you wish to test various implementations of an interface, or if the data being passed in to a function has lots of different requirements that need testing.

Let's demonstrate all this by adding another shape and testing it; a triangle.

Write the test first

Adding a new test for our new shape is very easy. Just add `{Triangle{12, 6}, 36.0},` to our list.

```
func TestArea(t *testing.T) {

    areaTests := []struct {
        shape Shape
        want  float64
```

```
        }{
            {Rectangle{12, 6}, 72.0},
            {Circle{10}, 314.1592653589793},
            {Triangle{12, 6}, 36.0},
        }

        for _, tt := range areaTests {
            got := tt.shape.Area()
            if got != tt.want {
                t.Errorf("got %g want %g", got, tt.want)
            }
        }

    }
```

Try to run the test

Remember, keep trying to run the test and let the compiler guide you toward a solution.

Write the minimal amount of code for the test to run and check the failing test output

```
./shapes_test.go:25:4: undefined: Triangle
```

We have not defined `Triangle` yet

```
    type Triangle struct {
        Base   float64
        Height float64
    }
```

Try again

```
./shapes_test.go:25:8: cannot use Triangle literal (type Triangle)
as type Shape in field value:
    Triangle does not implement Shape (missing Area method)
```

It's telling us we cannot use a `Triangle` as a shape because it does not have an `Area()` method, so add an empty implementation to get the test working

```
    func (t Triangle) Area() float64 {
        return 0
    }
```

Finally the code compiles and we get our error

```
shapes_test.go:31: got 0.00 want 36.00
```

Write enough code to make it pass

```
    func (t Triangle) Area() float64 {
        return (t.Base * t.Height) * 0.5
    }
```

And our tests pass!

Refactor

Again, the implementation is fine but our tests could do with some improvement.

When you scan this

```
{Rectangle{12, 6}, 72.0},
{Circle{10}, 314.1592653589793},
{Triangle{12, 6}, 36.0},
```

It's not immediately clear what all the numbers represent and you should be aiming for your tests to be easily understood.

So far you've only been shown syntax for creating instances of structs `MyStruct{val1, val2}` but you can optionally name the fields.

Let's see what it looks like

```
        {shape: Rectangle{Width: 12, Height: 6}, want: 72.0},
        {shape: Circle{Radius: 10}, want: 314.1592653589793},
        {shape: Triangle{Base: 12, Height: 6}, want: 36.0},
```

In Test-Driven Development by Example Kent Beck refactors some tests to a point and asserts:

> The test speaks to us more clearly, as if it were an assertion of truth, **not a sequence of operations**

(emphasis in the quote is mine)

Now our tests - rather, the list of test cases - make assertions of truth about shapes and their areas.

Make sure your test output is helpful

Remember earlier when we were implementing `Triangle` and we had the failing test? It printed `shapes_test.go:31: got 0.00 want 36.00`.

We knew this was in relation to `Triangle` because we were just working with it. But what if a bug slipped in to the system in one of 20 cases in the table? How would a developer know which case failed? This is not a great experience for the developer, they will have to manually look through the cases to find out which case actually failed.

We can change our error message into `%#v got %g want %g`. The `%#v` format string will print out our struct with the values in its field, so the developer can see at a glance the properties that are being tested.

To increase the readability of our test cases further, we can rename the `want` field into something more descriptive like `hasArea`.

One final tip with table driven tests is to use `t.Run` and to name the test cases.

By wrapping each case in a `t.Run` you will have clearer test output on failures as it will print the name of the case

```
--- FAIL: TestArea (0.00s)
    --- FAIL: TestArea/Rectangle (0.00s)
        shapes_test.go:33: main.Rectangle{Width:12, Height:6} got
72.00 want 72.10
```

And you can run specific tests within your table with `go test -run TestArea/Rectangle`.

Here is our final test code which captures this

```
        func TestArea(t *testing.T) {

            areaTests := []struct {
                name    string
                shape   Shape
                hasArea float64
            }{
                {name: "Rectangle", shape: Rectangle{Width: 12, Height: 6},
hasArea: 72.0},
                {name: "Circle", shape: Circle{Radius: 10}, hasArea:
314.1592653589793},
                {name: "Triangle", shape: Triangle{Base: 12, Height: 6},
hasArea: 36.0},
            }

            for _, tt := range areaTests {
                // using tt.name from the case to use it as the `t.Run` test
name
                t.Run(tt.name, func(t *testing.T) {
                    got := tt.shape.Area()
                    if got != tt.hasArea {
                        t.Errorf("%#v got %g want %g", tt.shape, got,
tt.hasArea)
                    }
                })

            }

        }
```

Wrapping up

This was more TDD practice, iterating over our solutions to basic mathematic problems and learning new language features motivated by our tests.

- Declaring structs to create your own data types which lets you bundle related data together and make the intent of your code clearer
- Declaring interfaces so you can define functions that can be used by different types (ad hoc polymorphism)
- Adding methods so you can add functionality to your data types and so you can implement interfaces
- Table driven tests to make your assertions clearer and your test suites easier to extend & maintain

This was an important chapter because we are now starting to define our own types. In statically typed languages like Go, being able to design your own types is essential for building software that is easy to understand, to piece together and to test.

Interfaces are a great tool for hiding complexity away from other parts of the system. In our case our test helper *code* did not need to know the exact shape it was asserting on, only how to "ask" for its area.

As you become more familiar with Go you will start to see the real strength of interfaces and the standard library. You'll learn about interfaces defined in the standard library that are used *everywhere* and by implementing them against your own types, you can very quickly re-use a lot of great functionality.

Maps

You can find all the code for this chapter here

In arrays & slices, you saw how to store values in order. Now, we will look at a way to store items by a `key` and look them up quickly.

Maps allow you to store items in a manner similar to a dictionary. You can think of the `key` as the word and the `value` as the definition. And what better way is there to learn about Maps than to build our own dictionary?

First, assuming we already have some words with their definitions in the dictionary, if we search for a word, it should return the definition of it.

Write the test first

In `dictionary_test.go`

```
package main

import "testing"

func TestSearch(t *testing.T) {
    dictionary := map[string]string{"test": "this is just a test"}

    got := Search(dictionary, "test")
    want := "this is just a test"

    if got != want {
        t.Errorf("got %q want %q given, %q", got, want, "test")
    }
}
```

Declaring a Map is somewhat similar to an array. Except, it starts with the `map` keyword and requires two types. The first is the key type, which is written inside the `[]`. The second is the value type, which goes right after the `[]`.

The key type is special. It can only be a comparable type because without the ability to tell if 2 keys are equal, we have no way to ensure that we are getting the correct value. Comparable types are explained in depth in the language spec.

The value type, on the other hand, can be any type you want. It can even be another map.

Everything else in this test should be familiar.

Try to run the test

By running `go test` the compiler will fail with `./dictionary_test.go:8:9: undefined: Search`.

Write the minimal amount of code for the test to run and check the output

In `dictionary.go`

```
package main

func Search(dictionary map[string]string, word string) string {
    return ""
}
```

Your test should now fail with a *clear error message*

```
dictionary_test.go:12: got '' want 'this is just a test' given,
'test'.
```

Write enough code to make it pass

```
func Search(dictionary map[string]string, word string) string {
    return dictionary[word]
}
```

Getting a value out of a Map is the same as getting a value out of Array `map[key]`.

Refactor

```
func TestSearch(t *testing.T) {
    dictionary := map[string]string{"test": "this is just a test"}

    got := Search(dictionary, "test")
    want := "this is just a test"

    assertStrings(t, got, want)
}

func assertStrings(t testing.TB, got, want string) {
    t.Helper()

    if got != want {
        t.Errorf("got %q want %q", got, want)
    }
}
```

I decided to create an `assertStrings` helper to make the implementation more general.

Using a custom type

We can improve our dictionary's usage by creating a new type around map and making `Search` a method.

In `dictionary_test.go`:

```
func TestSearch(t *testing.T) {
    dictionary := Dictionary{"test": "this is just a test"}

    got := dictionary.Search("test")
    want := "this is just a test"
```

```
    assertStrings(t, got, want)
}
```

We started using the `Dictionary` type, which we have not defined yet. Then called `Search` on the `Dictionary` instance.

We did not need to change `assertStrings`.

In `dictionary.go`:

```
type Dictionary map[string]string

func (d Dictionary) Search(word string) string {
    return d[word]
}
```

Here we created a `Dictionary` type which acts as a thin wrapper around `map`. With the custom type defined, we can create the `Search` method.

Write the test first

The basic search was very easy to implement, but what will happen if we supply a word that's not in our dictionary?

We actually get nothing back. This is good because the program can continue to run, but there is a better approach. The function can report that the word is not in the dictionary. This way, the user isn't left wondering if the word doesn't exist or if there is just no definition (this might not seem very useful for a dictionary. However, it's a scenario that could be key in other usecases).

```
func TestSearch(t *testing.T) {
    dictionary := Dictionary{"test": "this is just a test"}

    t.Run("known word", func(t *testing.T) {
        got, _ := dictionary.Search("test")
        want := "this is just a test"

        assertStrings(t, got, want)
    })

    t.Run("unknown word", func(t *testing.T) {
        _, err := dictionary.Search("unknown")
        want := "could not find the word you were looking for"

        if err == nil {
            t.Fatal("expected to get an error.")
        }

        assertStrings(t, err.Error(), want)
    })
}
```

The way to handle this scenario in Go is to return a second argument which is an `Error` type.

Notice that as we've seen in the pointers and error section here in order to assert the error message we first check that the error is not `nil` and then use `.Error()` method to get the string which we can then pass to the assertion.

Try and run the test

This does not compile

```
./dictionary_test.go:18:10: assignment mismatch: 2 variables but 1
values
```

Write the minimal amount of code for the test to run and check the output

```
        func (d Dictionary) Search(word string) (string, error) {
            return d[word], nil
        }
```

Your test should now fail with a much clearer error message.

```
dictionary_test.go:22: expected to get an error.
```

Write enough code to make it pass

```
        func (d Dictionary) Search(word string) (string, error) {
            definition, ok := d[word]
            if !ok {
                return "", errors.New("could not find the word you were
looking for")
            }

            return definition, nil
        }
```

In order to make this pass, we are using an interesting property of the map lookup. It can return 2 values. The second value is a boolean which indicates if the key was found successfully.

This property allows us to differentiate between a word that doesn't exist and a word that just doesn't have a definition.

Refactor

```
        var ErrNotFound = errors.New("could not find the word you were
looking for")

        func (d Dictionary) Search(word string) (string, error) {
            definition, ok := d[word]
            if !ok {
                return "", ErrNotFound
            }

            return definition, nil
        }
```

We can get rid of the magic error in our `Search` function by extracting it into a variable. This will also allow us to have a better test.

```
        t.Run("unknown word", func(t *testing.T) {
            _, got := dictionary.Search("unknown")
            if got == nil {
                t.Fatal("expected to get an error.")
            }
            assertError(t, got, ErrNotFound)
```

```
})

func assertError(t testing.TB, got, want error) {
    t.Helper()

    if got != want {
        t.Errorf("got error %q want %q", got, want)
    }
}
```

By creating a new helper we were able to simplify our test, and start using our `ErrNotFound` variable so our test doesn't fail if we change the error text in the future.

Write the test first

We have a great way to search the dictionary. However, we have no way to add new words to our dictionary.

```
func TestAdd(t *testing.T) {
    dictionary := Dictionary{}
    dictionary.Add("test", "this is just a test")

    want := "this is just a test"
    got, err := dictionary.Search("test")
    if err != nil {
        t.Fatal("should find added word:", err)
    }

    assertStrings(t, got, want)
}
```

In this test, we are utilizing our `Search` function to make the validation of the dictionary a little easier.

Write the minimal amount of code for the test to run and check output

In `dictionary.go`

```
func (d Dictionary) Add(word, definition string) {
}
```

Your test should now fail

```
dictionary_test.go:31: should find added word: could not find the
word you were looking for
```

Write enough code to make it pass

```
func (d Dictionary) Add(word, definition string) {
    d[word] = definition
}
```

Adding to a map is also similar to an array. You just need to specify a key and set it equal to a value.

Pointers, copies, et al

An interesting property of maps is that you can modify them without passing as an address to it (e.g `&myMap`)

This may make them *feel* like a "reference type", but as Dave Cheney describes they are not.

> A map value is a pointer to a runtime.hmap structure.

So when you pass a map to a function/method, you are indeed copying it, but just the pointer part, not the underlying data structure that contains the data.

A gotcha with maps is that they can be a `nil` value. A `nil` map behaves like an empty map when reading, but attempts to write to a `nil` map will cause a runtime panic. You can read more about maps here.

Therefore, you should never initialize a nil map variable:

```
var m map[string]string
```

Instead, you can initialize an empty map or use the `make` keyword to create a map for you:

```
var dictionary = map[string]string{}

// OR

var dictionary = make(map[string]string)
```

Both approaches create an empty `hash map` and point `dictionary` at it. Which ensures that you will never get a runtime panic.

Refactor

There isn't much to refactor in our implementation but the test could use a little simplification.

```
func TestAdd(t *testing.T) {
    dictionary := Dictionary{}
    word := "test"
    definition := "this is just a test"

    dictionary.Add(word, definition)

    assertDefinition(t, dictionary, word, definition)
}

func assertDefinition(t testing.TB, dictionary Dictionary, word,
definition string) {
    t.Helper()

    got, err := dictionary.Search(word)
    if err != nil {
        t.Fatal("should find added word:", err)
    }
    assertStrings(t, got, definition)
}
```

We made variables for word and definition, and moved the definition assertion into its own helper function.

Our `Add` is looking good. Except, we didn't consider what happens when the value we are trying to add already exists!

Map will not throw an error if the value already exists. Instead, they will go ahead and overwrite the value with the newly provided value. This can be convenient in practice, but makes our function name less than accurate. `Add` should not modify existing values. It should only add new words to our dictionary.

Write the test first

```
func TestAdd(t *testing.T) {
    t.Run("new word", func(t *testing.T) {
        dictionary := Dictionary{}
        word := "test"
        definition := "this is just a test"

        err := dictionary.Add(word, definition)

        assertError(t, err, nil)
        assertDefinition(t, dictionary, word, definition)
    })

    t.Run("existing word", func(t *testing.T) {
        word := "test"
        definition := "this is just a test"
        dictionary := Dictionary{word: definition}
        err := dictionary.Add(word, "new test")

        assertError(t, err, ErrWordExists)
        assertDefinition(t, dictionary, word, definition)
    })
}
```

For this test, we modified `Add` to return an error, which we are validating against a new error variable, `ErrWordExists`. We also modified the previous test to check for a `nil` error.

Try to run test

The compiler will fail because we are not returning a value for `Add`.

```
./dictionary_test.go:30:13: dictionary.Add(word, definition) used as
value
./dictionary_test.go:41:13: dictionary.Add(word, "new test") used as
value
```

Write the minimal amount of code for the test to run and check the output

In `dictionary.go`

```
var (
    ErrNotFound   = errors.New("could not find the word you were
looking for")
    ErrWordExists = errors.New("cannot add word because it already
exists")
)

func (d Dictionary) Add(word, definition string) error {
    d[word] = definition
    return nil
}
```

Now we get two more errors. We are still modifying the value, and returning a `nil` error.

```
dictionary_test.go:43: got error '%!q(<nil>)' want 'cannot add word
because it already exists'
dictionary_test.go:44: got 'new test' want 'this is just a test'
```

Write enough code to make it pass

```
func (d Dictionary) Add(word, definition string) error {
    _, err := d.Search(word)

    switch err {
    case ErrNotFound:
        d[word] = definition
    case nil:
        return ErrWordExists
    default:
        return err
    }

    return nil
}
```

Here we are using a `switch` statement to match on the error. Having a `switch` like this provides an extra safety net, in case `Search` returns an error other than `ErrNotFound`.

Refactor

We don't have too much to refactor, but as our error usage grows we can make a few modifications.

```
const (
    ErrNotFound   = DictionaryErr("could not find the word you were
looking for")
    ErrWordExists = DictionaryErr("cannot add word because it
already exists")
)

type DictionaryErr string

func (e DictionaryErr) Error() string {
    return string(e)
}
```

We made the errors constant; this required us to create our own `DictionaryErr` type which implements the `error` interface. You can read more about the details in this excellent article by Dave Cheney. Simply put, it makes the errors more reusable and immutable.

Next, let's create a function to `Update` the definition of a word.

Write the test first

```
func TestUpdate(t *testing.T) {
    word := "test"
    definition := "this is just a test"
    dictionary := Dictionary{word: definition}
    newDefinition := "new definition"
```

```
    dictionary.Update(word, newDefinition)

    assertDefinition(t, dictionary, word, newDefinition)
}
```

`Update` is very closely related to `Add` and will be our next implementation.

Try and run the test

```
./dictionary_test.go:53:2: dictionary.Update undefined (type
Dictionary has no field or method Update)
```

Write minimal amount of code for the test to run and check the failing test output

We already know how to deal with an error like this. We need to define our function.

```
func (d Dictionary) Update(word, definition string) {}
```

With that in place, we are able to see that we need to change the definition of the word.

```
dictionary_test.go:55: got 'this is just a test' want 'new
definition'
```

Write enough code to make it pass

We already saw how to do this when we fixed the issue with `Add`. So let's implement something really similar to `Add`.

```
func (d Dictionary) Update(word, definition string) {
    d[word] = definition
}
```

There is no refactoring we need to do on this since it was a simple change. However, we now have the same issue as with `Add`. If we pass in a new word, `Update` will add it to the dictionary.

Write the test first

```
t.Run("existing word", func(t *testing.T) {
    word := "test"
    definition := "this is just a test"
    dictionary := Dictionary{word: definition}
    newDefinition := "new definition"

    err := dictionary.Update(word, newDefinition)

    assertError(t, err, nil)
    assertDefinition(t, dictionary, word, newDefinition)
})

t.Run("new word", func(t *testing.T) {
    word := "test"
    definition := "this is just a test"
    dictionary := Dictionary{}
```

```
        err := dictionary.Update(word, definition)

        assertError(t, err, ErrWordDoesNotExist)
    })
```

We added yet another error type for when the word does not exist. We also modified `Update` to return an `error` value.

Try and run the test

```
./dictionary_test.go:53:16: dictionary.Update(word, newDefinition)
used as value
./dictionary_test.go:64:16: dictionary.Update(word, definition) used
as value
./dictionary_test.go:66:23: undefined: ErrWordDoesNotExist
```

We get 3 errors this time, but we know how to deal with these.

Write the minimal amount of code for the test to run and check the failing test output

```
    const (
        ErrNotFound         = DictionaryErr("could not find the word you
were looking for")
        ErrWordExists       = DictionaryErr("cannot add word because it
already exists")
        ErrWordDoesNotExist = DictionaryErr("cannot perform operation on
word because it does not exist")
    )

    func (d Dictionary) Update(word, definition string) error {
        d[word] = definition
        return nil
    }
```

We added our own error type and are returning a `nil` error.

With these changes, we now get a very clear error:

```
dictionary_test.go:66: got error '%!q(<nil>)' want 'cannot update
word because it does not exist'
```

Write enough code to make it pass

```
    func (d Dictionary) Update(word, definition string) error {
        _, err := d.Search(word)

        switch err {
        case ErrNotFound:
            return ErrWordDoesNotExist
        case nil:
            d[word] = definition
        default:
            return err
        }

        return nil
```

```
}
```

This function looks almost identical to `Add` except we switched when we update the `dictionary` and when we return an error.

Note on declaring a new error for Update

We could reuse `ErrNotFound` and not add a new error. However, it is often better to have a precise error for when an update fails.

Having specific errors gives you more information about what went wrong. Here is an example in a web app:

> You can redirect the user when `ErrNotFound` is encountered, but display an error message when `ErrWordDoesNotExist` is encountered.

Next, let's create a function to `Delete` a word in the dictionary.

Write the test first

```
func TestDelete(t *testing.T) {
    word := "test"
    dictionary := Dictionary{word: "test definition"}

    dictionary.Delete(word)

    _, err := dictionary.Search(word)
    assertError(t, err, ErrNotFound)
}
```

Our test creates a `Dictionary` with a word and then checks if the word has been removed.

Try to run the test

By running `go test` we get:

```
./dictionary_test.go:74:6: dictionary.Delete undefined (type
Dictionary has no field or method Delete)
```

Write the minimal amount of code for the test to run and check the failing test output

```
func (d Dictionary) Delete(word string) {

}
```

After we add this, the test tells us we are not deleting the word.

```
dictionary_test.go:78: got error '%!q(<nil>)' want 'could not find
the word you were looking for'
```

Write enough code to make it pass

```
func (d Dictionary) Delete(word string) {
    delete(d, word)
```

```
}
```

Go has a built-in function `delete` that works on maps. It takes two arguments and returns nothing. The first argument is the map and the second is the key to be removed.

Refactor

There isn't much to refactor, but we can implement the same logic from `Update` to handle cases where word doesn't exist.

```
func TestDelete(t *testing.T) {
    t.Run("existing word", func(t *testing.T) {
        word := "test"
        dictionary := Dictionary{word: "test definition"}

        err := dictionary.Delete(word)

        assertError(t, err, nil)

        _, err = dictionary.Search(word)

        assertError(t, err, ErrNotFound)
    })

    t.Run("non-existing word", func(t *testing.T) {
        word := "test"
        dictionary := Dictionary{}

        err := dictionary.Delete(word)

        assertError(t, err, ErrWordDoesNotExist)
    })
}
```

Try to run test

The compiler will fail because we are not returning a value for `Delete`.

```
./dictionary_test.go:77:10: dictionary.Delete(word) (no value) used
as value
./dictionary_test.go:90:10: dictionary.Delete(word) (no value) used
as value
```

Write enough code to make it pass

```
func (d Dictionary) Delete(word string) error {
    _, err := d.Search(word)

    switch err {
    case ErrNotFound:
        return ErrWordDoesNotExist
    case nil:
        delete(d, word)
    default:
        return err
    }

    return nil
}
```

We are again using a switch statement to match on the error when we attempt to delete a word that doesn't exist.

Wrapping up

In this section, we covered a lot. We made a full CRUD (Create, Read, Update and Delete) API for our dictionary. Throughout the process we learned how to:

- Create maps
- Search for items in maps
- Add new items to maps
- Update items in maps
- Delete items from a map
- Learned more about errors
  - How to create errors that are constants
  - Writing error wrappers